

| Tipos Abstratos De Dados

DCC024

João Eduardo Montandon

```
Person x;
```

É possível `x` referenciar algum tipo diferente de `Person`? 🤔

Em Java, uma referência pode apontar para outro tipo, *desde que esteja estruturalmente relacionado ao tipo declarado*.

Afinal, como linguagem orientada a objetos, Java suporta polimorfismo de subtipos!

| Interfaces

- Uma **interface** é uma lista de declaração de métodos.
- Propósito: estabelecer um contrato do que a classe é obrigada a implementar.

| Exemplo Clássico

```
public interface FormaGeometrica {  
    public double calcularArea();  
    public double calcularPerimetro();  
}
```

- Uma classe pode declarar que *implementa* uma ou mais interfaces.
- Isto obriga a classe a implementar os métodos definidos em cada interface.
- Esses métodos devem ser públicos.

```
public class Retangulo implements FormaGeometrica {
    private int base;
    private int altura;
    public Retangulo(int base, int altura) {
        this.base = base;
        this.altura = altura;
    }
    @Override
    public double calcularArea() {
        return base * altura;
    }
    @Override
    public double calcularPerimetro() {
        return 2 * (base + altura);
    }
}
```

```
public class Circulo implements FormaGeometrica {
    private double raio;
    public Circulo(double raio) {
        this.raio = raio;
    }
    @Override
    public double calcularArea() {
        return Math.PI * Math.pow(this.raio, 2);
    }
    @Override
```

```
    public double calcularPerimetro() {  
        return 2 * Math.PI * raio;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        FormaGeometrica forma;  
        forma = new Retangulo(2, 4);  
        System.out.println(retangulo.calcularArea());  
  
        System.out.println(retangulo.calcularPerimetro());  
  
        forma = new Circulo(3.12);  
        System.out.println(circulo.calcularArea());  
        System.out.println(circulo.calcularPerimetro());  
    }  
}
```

| Por Que Interfaces?

- Pode ser aproveitada por muitas classes.
- Uma classe pode adotar muitas interfaces.
- Pode ser referenciada como tipo, portanto sujeita a polimorfismo!

| Exemplo Interessante

- `MyCollection` é uma interface para uma coleção de strings.
- Ações:
 1. adicionar

2. remover
3. checar se está vazia

```
/**
 * Uma interface para uma coleção de Strings.
 *
 * @author João Eduardo Montandon
 */
public interface MyCollection {

    /**
     * Adiciona uma String a lista.
     * @param item elemento a ser adicionado.
     */
    void add(String item);

    /**
     * Testa se a lista está vazia.
     * @return true se não houver qualquer elemento
     * na lista, false caso contrário.
     */
    boolean isEmpty();

    /**
     * Remove um elemento da lista, retornando-o como
     * resultado da chamada.
     * @return item que foi removido da lista.
     */
    String remove();
}
```

| Observações

- Comentários são particularmente interessantes em interfaces.
 - Não há código no corpo dos métodos, apenas sua assinatura.
 - Servem como guia básico para quem desejar implementar a interface.
 - Dizem o que é, não como é!
-

```
public class Node {
    private String data;
    private Node next;
    public Node(String data, Node next) {
        this.data = data;
        this.next = next;
    }
    public String getValue() {
        return this.data;
    }
    public Node getNext() {
        return this.next;
    }
}
```

```
public class Stack implements MyCollection {
    private Node top = null;

    public void add(String data) {
        this.top = new Node(data, top);
    }
    public String remove() {
        String value = this.top.getValue();
        this.top = this.top.getNext();

        return value;
    }
    public boolean isEmpty() {
```

```
        return this.top == null;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        MyCollection s = new Stack();
        s.add("O rato");
        s.add("roeu");
        s.add("a roupa");
        s.add("do rei");
        s.add("de roma");

        while(!s.isEmpty())
            System.out.println(s.remove());
    }
}
```

| Estendendo Classes

- A extensão via `implements` é permitido apenas para interfaces.
 - Caso queira estender o comportamento de uma classe, deve-se usar `extends`
 - Serão herdados *todos os membros* da classe estendida.
 - Também habilita uso de polimorfismo.
-

```
public class PeekStack extends Stack {
    public String peek() {
        String s = this.remove();
        this.add(s);
        return s;
    }
}
```

```
}  
}
```

Não teria um jeito mais fácil? 🤔

```
public class Stack implements MyCollection {  
    protected Node top = null;  
    ...  
}  
  
public class PeekStack extends Stack {  
    public String peek() {  
        return this.top.getValue();  
    }  
}
```

Agora podemos "bisbilhotar" o topo da pilha, sem efetivamente modificá-la.

```
Stack s1 = new PeekStack();  
PeekStack s2 = new PeekStack();  
s1.add("olá");  
s2.add("mundo");  
System.out.println(s2.peek());
```

| Cadeia De Herança

- Toda subclasse pode estender no máximo uma única superclasse (Herança Simples)
- Mas uma subclasse pode herdar o comportamento de mais de uma superclasse.

Como? 🤔

```
public class A { ... }  
public class B extends A { ... }  
public class C extends B { ... }
```

`C` herda diretamente `B`, e `A` indiretamente.

| A Classe Object

- Em Java, existe efetivamente uma única classe base chamada `Object`.
- Todas as classes da linguagem herdam `Object` direta ou indiretamente.

Qual o propósito de `Object`? 🤔

Esse código compila? 🤔

```
System.out.println(1);  
System.out.println("String");  
System.out.println(new Stack());
```

```
1  
String  
Stack@251a69d7
```

O que aconteceu? 🤔

- Parte 1: [PrintStream \(Java Platform SE 8\)](#).

- Parte 2: [String \(Java Platform SE 8\)](#).

Bem, se `println(Obj)` invoca o método `toString()` da classe `Object`...

```
public class Stack {  
    ...  
    @Override  
    public String toString() {  
        Node topCopy = this.top;  
        String result = "[";  
        for(Node curr = this.top;  
            curr != null;  
            curr = curr.getNext())  
            result += curr.getValue() + ",";  
        result += "];"  
        return result;  
    }  
}
```

```
Stack s = new Stack();  
s.add("1");  
s.add("2");  
s.add("3");  
System.out.println(s);
```

```
[3,2,1,]
```

| Uma Reflexão Sobre Herança

| Por Que Utilizar?

- Quando escrevemos os mesmos comandos várias vezes...
 - Isso deveria estar em um método?
 - Quando escrevemos os mesmos métodos várias vezes...
 - Isso deveria estar em uma superclasse?
 - O truque está em perceber quando é necessário ter uma classe base.
-

| Devo Utilizar? 🤔

Lembre-se do que estudamos na aula de [Tipos](#).

Um subtipo é um subconjunto de valores, mas suporta um superconjunto de operações.

Ao projetar uma hierarquia de classes, tenha em mente o seguinte:

1. Uma subclasse também **é uma** superclasse.
 2. Herança diz respeito **as funcionalidades** que serão **herdadas**.
-

Priorize composição em detrimento a herança.^[1]

| Herança Múltipla

Não é suportado! Consegue imaginar o por quê? 🤔

Vale a pena?? 🤔

| Generics

- A interface `MyCollection` suporta apenas valores do tipo `String`.
- Como armazenar outros valores? 🤔

| Nos Tempos Antigos

Se toda e qualquer classe herda de `Object` ...

```
public interface ObjectCollection {  
    void add(Object obj);  
    boolean isEmpty();  
    Object remove();  
}
```

Quais os problemas que isso pode trazer? 🤔

- Verificação apenas em tempo de execução.
- Coleção poderia armazenar qualquer coisa.

| Nos "Novos" Tempos

- Em 2004, a linguagem foi estendida para suportar tipos variáveis.
- Com isso, a linguagem passou a suportar [polimorfismo paramétrico](#).

```
public interface MyCollection<T> {  
    void add(T item);  
    boolean isEmpty();  
}
```

```
    T remove();  
}
```

```
public class Node<T> {  
    private T data;  
    private Node next;  
    public Node(T data, Node next) {  
        this.data = data;  
        this.next = next;  
    }  
    public T getValue() {  
        return this.data;  
    }  
    public Node getNext() {  
        return this.next;  
    }  
}
```

```
public class Stack<T> implements MyCollection<T> {  
    protected Node<T> top = null;  
    public void add(T data) {  
        this.top = new Node(data, top);  
    }  
    public T remove() {  
        T value = this.top.getValue();  
        this.top = this.top.getNext();  
  
        return value;  
    }  
    public boolean isEmpty() {  
        return this.top == null;  
    }  
    @Override
```

```
public String toString() {
    Node topCopy = this.top;
    String result = "[";
    for(Node curr = this.top; curr != null; curr =
curr.getNext())
        result += curr.getValue() + ",";
    result += "]";
    return result;
}
}
```

```
public class Main {
    public static void main(String[] args) {
        Stack<Integer> intStack = new Stack<>();
        Stack<String> strStack = new Stack<>();

        intStack.add(1);
        intStack.add(2);
        intStack.add(3);
        strStack.add("a");
        strStack.add("b");
        strStack.add("c");

        System.out.println(intStack);
        System.out.println(strStack);
    }
}
```

| Considerações Finais

- Java apresenta dois mecanismos para relacionamento total.
- **interface**: Habilita polimorfismo de subtipagem entre classes que devem implementar o mesmo contrato.

- **extends** : Habilita herança e polimorfismo de subtipagem entre duas classes.
 - Não suporta herança múltipla.
 - Generics: mecanismo introduzido em 2004 para permitir polimorfismo paramétrico.
-
-

1. [Composition over inheritance - Wikipedia](#) ↩